

챗터 5. 사서가 답해준다. LCEL 파이프라인

이번 챗터가 끝나면

- **LCEL**(LangChain Expression Language) 파이프 연산자로 Retriever → Prompt → LLM → Parser를 조립합니다
- **출처 강제(Source Grounding)** 프롬프트로 환각을 줄이고 답변에 출처를 항상 붙입니다
- **WindowMemory(k=5)** 로 최근 5턴 멀티턴 대화를 유지합니다
- 채팅 UI에서 커넥트HR 에이전트의 전신인 RAG Q&A 엔진과 실제로 대화해 봅니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex05
```

파일 구조는 다음과 같습니다.

ex05 디렉토리

```
ex05/
├── run.py                # [참고] 서버 실행 진입점
├── .env                  # [참고] 환경 변수
├── data/
│   ├── docs/            # [참고] 원본 문서 (ex04와 동일 샘플)
│   ├── markdown/        # [참고] 파싱 결과
│   └── chroma_db/        # [참고] 벡터 DB (첫 실행 시 자동 생성)
├── src/
│   ├── rag_chain.py      # [실습] LCEL 파이프라인 + 출처 강제
│   ├── conversation.py  # [실습] WindowMemory(k=5)
│   ├── llm_factory.py    # [참고] Ollama/OpenAI 분기
│   ├── vectorstore.py    # [참고] Retriever 생성
│   └── session_manager.py # [참고] 세션별 Memory + TTL
```

```
├─ static/                # [참고] 채팅 UI CSS/JS
└─ app/
    ├── main.py           # [참고] FastAPI 진입점
    ├── chat_api.py       # [설명] POST /api/chat
    └── session.py        # [참고] 세션 쿠키
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```
cd ex05
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
ollama pull deepseek-r1:8b
pip install -r requirements.txt
```

터미널 환경 구성. Windows

```
cd ex05
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
ollama pull deepseek-r1:8b
pip install -r requirements.txt
```

.env 핵심 상수입니다. 코드를 보기 전에 역할부터 파악해 두면 흐름이 잘 읽힙니다.

상수	기본값	역할
LLM_PROVIDER	ollama	사용할 LLM 제공자 (ollama 또는 openai)
OLLAMA_MODEL	deepseek-r1:8b	Ollama에서 쓸 모델 이름
EMBEDDING_MODEL	jhgan/ko-sroberta-multitask	문서 임베딩 모델. 챗터 4와 같은 값이어야 검색이 맞음
CHROMA_PERSIST_DIR	./data/chroma_db	벡터 DB 저장 경로
RETRIEVER_TOP_K	5	질문당 검색해서 가져올 관련 문서 조각 개수

CONVERSATION_WINDOW_SIZE

5

프롬프트에 포함할 최근 대화 턴 수

3. 사용할 라이브러리

패키지	역할
<code>langchain</code>	체인 조립 프레임워크
<code>langchain-community</code>	HuggingFaceEmbeddings 등 커뮤니티 통합
<code>langchain-ollama</code>	Ollama LLM 연결
<code>langchain-chroma</code>	ChromaDB Retriever 래퍼
<code>fastapi</code> · <code>uvicorn</code>	웹 API + ASGI 서버

4. 실습 순서

1. `src/rag_chain.py` . LCEL 파이프라인 + 출처 강제 프롬프트
2. `src/conversation.py` . WindowMemory(k=5)
3. `python run.py` . 서버 실행 후 브라우저 `http://localhost:8000/chat` 에서 대화

5.1 "그냥 답을 알려줘"



그림 5-1. 사서가 답해준다. 책을 찾아서 읽고, 출처와 함께 답해 줍니다

수요일 점심 직후. 사무실 라디에이터가 한 번 울컥 소리를 내고 다시 잠잠해졌습니다.

챗터 4에서 벡터 검색까지 돌렸고, 동료에게 CLI 사용법을 알려 줬습니다. 점심 먹고 돌아오니 그 동료가 또 의자를 돌려 제 쪽을 봤습니다.

동료 : "병가 쓸 때 증빙 서류 필요해요?"

동료 : "연차 며칠 남았는지 어떻게 확인해요?"

CLI가 켜진 터미널을 그대로 보여 줬습니다.

벡터 검색 결과

1 HR_취업규칙_v1.0 (p.3)

유사도 87.2%

"제15조(병가) 질병이나 부상으로 인하여 직무를 수행할 수 없을 때에는..."

2 HR_취업규칙_v1.0 (p.4)

유사도 81.5%

"병가 기간이 3일 이상인 경우에는 의사의 진단서를..."

동료 : "이걸 제가 읽어요? 그냥 답을 알려주세요."

맞다. 사람들은 문서 조각을 원하는 게 아니라 **답변**을 원한다.

벡터 검색은 **재료를 찾아 주는 일**이지 **요리를 해 주는 일**이 아닙니다. 이번 챗터에서는 그 "요리"를 해 주는 **RAG Q&A 엔진**을 만듭니다. 질문하면 문서를 검색하고, 읽어서, 자연어로 답해 주는 시스템입니다.

5.2 사서가 일하는 네 단계

지금까지 만든 벡터 검색은 **도서관의 검색 시스템**과 비슷합니다. "한국 역사"를 검색하면 관련 책이 어디 있는지 알려주지만, 책을 꺼내서 읽고 정리해 주지는 않습니다.

우리에게 필요한 건 **사서**입니다. 사서에게 "조선시대 과거 제도가 뭐야?"라고 물으면 네 단계가 일어납니다.

1. **질문을 듣는다.** "조선 과거 제도"가 핵심이군
2. **서가에서 책을 찾는다.** 한국사 개론 3장 근처를 꺼냄
3. **읽고 답변을 정리한다.** "문과, 무과, 잡과 세 종류가 있었습니다"
4. **출처를 알려준다.** "한국사 개론 3장에 나와 있어요"

이 네 단계가 바로 RAG Q&A 파이프라인입니다. 챗터 4가 2번까지였다면, 이번 챗터가 **3, 4번을** 추가합니다.



그림 5-2. RAG Q&A 흐름. 챗터 4까지가 1~2번이었고, 이번 챗터에서 3~4번을 추가합니다

5.3 LCEL 조립과 출처 강제(rag_chain.py)

검색 → 프롬프트 → LLM → 파싱. 각 단계를 직접 이어 붙여도 되지만, **LangChain**이 이걸 부품화해 줍니다. 조립 방식은 **파이프 연산자**(`|`). 이를 **LCEL(LangChain Expression Language)** 이라 부릅니다.

그림 5-3. LCEL 파이프라인 구조. 앞 블록의 출력이 `|`를 타고 다음 블록의 입력이 됩니다

파이프 연산자(`|`) 는 앞 블록의 출력을 뒤 블록의 입력으로 흘려보내는 역할을 합니다. 유닉스 셸의 `ls | grep` 파이프와 같은 발상입니다. 덕분에 Retriever, Prompt, LLM, Parser 어느 블록이든 **같은 인터페이스만 지키면 자리 바꿔 끼울 수 있습니다**. LLM을 DeepSeek에서 GPT-4o로 갈아 끼울 때도 체인 구조는 건드리지 않고 LLM 블록만 교체하면 됩니다. 챗터 8~10에서 튜닝할 때 이 구조의 힘이 드러납니다.

출처 강제(Source Grounding)

옆 자리에서 모니터를 훑듯 본 팀장이 한마디 던졌습니다.

팀장 : "제공 문서에서만이라고 못 박아 뒀요. 안 그러면 또 끼워서 답해요."

챗터 1에서 맛본 **그라운드링**의 강한 버전입니다. 그때는 "참고 정보에서만 답하세요" 한 줄이었지만, 실서비스에서는 그걸로 부족합니다. LLM이 여전히 "문서에서 찾을 수 없으면" 외부 지식을 슬쩍 섞어 지어냅니다.

이번 챗터는 규칙을 **네 개로 늘립니다**. (1) 제공된 문서에서만, (2) 없으면 "확인되지 않습니다", (3) 답변 끝에 **[출처: 파일명]**, (4) 추측 금지. `ex05/src/rag_chain.py` 상단에 상수 `RAG_SYSTEM_PROMPT` 로 박혀 있습니다.

실습(rag_chain.py)

설명 ex05/src/rag_chain.py. 출처 강제 프롬프트 (발췌)

```
RAG_SYSTEM_PROMPT = """당신은 커넥트HR 사내 문서 Q&A 비서입니다.
아래에 제공된 문서(Context)만 사용하여 질문에 답변하십시오.
```

규칙:

1. 반드시 제공된 문서에서만 근거를 찾아 답변하십시오.
2. 문서에서 답을 찾을 수 없으면 "해당 내용은 제공된 문서에서 확인되지 않습니다."라고 답하십시오.
3. 답변 마지막에 근거 문서명을 반드시 명시하십시오. 형식: [출처: 문서명]
4. 추측이나 외부 지식을 사용하지 마시오.

```
Context (제공된 문서):
{context}
```

```
이전 대화:
{history}
"""
```

`{context}` 에는 검색된 문서가, `{history}` 에는 이전 대화가 자동으로 채워집니다. 규칙 2번이 **환각 방지**(모르면 모른다고 답하라), 규칙 3번이 **출처 명시**를 담당합니다. 같은 파일에 `RAG_HUMAN_PROMPT = "질문: {question}"` 상수도 함께 선언돼 있어, 뒤에서 `("human", RAG_HUMAN_PROMPT)` 자리에 그대로 끼워 넣습니다. `_format_docs()` 와 `build_rag_chain()` 두 함수에 TODO가 있습니다. TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 ex05/src/rag_chain.py. `_format_docs`

```
def _format_docs(docs):
    # TODO: 검색된 Document를 "[문서 N] 출처: ..." 텍스트로 변환
    # 1. 검색된 Document 리스트를 프롬프트에 넣을 텍스트로 변환
```

```

parts = []
for i, doc in enumerate(docs, start=1):
    source = doc.metadata.get("source", "알 수 없음")
    page = doc.metadata.get("page", "-")
    parts.append(f"[문서 {i}] 출처: {source} (p.{page})\n{doc.page_content}")
return "\n\n".join(parts)

```

각 문서에 `[출처: 파일명 (p.N)]` 라벨을 붙여 한 문자열로 합칩니다. 이 결과가 프롬프트 변수 `{context}` 로 들어갑니다. 이어서 본체 조립. 챗터 1에서 RetrievalQA가 내부에서 알아서 해주던 검색·프롬프트·LLM 호출을, 이번에는 Retriever부터 하나씩 꺼내서 파이프로 직접 조립합니다.

실습 1 ex05/src/rag_chain.py. build_rag_chain

```

def build_rag_chain():
    # TODO: build_llm()으로 LLM 생성 ~ (chain, retriever) 튜플 반환
    # 1. LLM 인스턴스 생성 (llm_factory.py에서 Ollama/OpenAI 선택)
    llm = build_llm()
    # 2. ChromaDB에서 문서를 검색하는 Retriever 생성
    retriever = build_retriever()

    # 3. 시스템 프롬프트 + 사용자 프롬프트 조립
    prompt = ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_PROMPT),
        ("human", RAG_HUMAN_PROMPT),
    ])

    # 4. LCEL 파이프라인 체인 조립. 질문→검색→포맷→프롬프트→LLM→텍스트
    chain = (
        {
            "context": itemgetter("question") | retriever | _format_docs,
            "history": itemgetter("history"),
            "question": itemgetter("question"),
        }
        | prompt
        | llm
        | StrOutputParser()
    )

    # 5. 체인과 검색기를 함께 반환
    return chain, retriever

```

파이프 연산자(`|`)가 각 단계의 출력을 다음 단계의 입력으로 넘깁니다. `itemgetter("question")` 은 입력 dict에서 `question` 키만 꺼내는 호출가능 객체입니다. 등장한 LangChain 부품 세 개를 한 줄씩 정리하면 아래와 같습니다.

부품	역할
<code>ChatPromptTemplate.from_messages([...])</code>	<code>system</code> / <code>human</code> 메시지로 프롬프트 조립. <code>{context}</code> , <code>{question}</code> 같은 변수는 체인 실행 시 자동으로 채워짐
<code>build_llm()</code>	<code>.env</code> 의 <code>LLM_PROVIDER</code> 에 따라 <code>ChatOllama</code> 또는 <code>ChatOpenAI</code> 인스턴스를 만들어 줌 (<code>llm_factory.py</code>)
<code>StrOutputParser()</code>	LLM이 돌려주는 <code>AIMessage</code> 객체에서 <code>.content</code> 문자열만 꺼내는 최종 파서

왜 **chain**과 **retriever**를 같이 반환할까요? 체인은 최종 답변 텍스트만 돌려주지만, 출처 표시용으로 검색된 원본 **Document** 도 따로 필요합니다. `ex05/app/chat_api.py` 가 `retriever.invoke(question)` 을 한 번 더 호출해 출처 목록을 만들어 응답 JSON에 붙입니다.

5.4 WindowMemory. conversation.py

도서관 방문자가 사서에게 이어서 묻습니다.

방문자 : "한국 역사 관련 책 어디 있어요?"

사서 : "2층 인문학 서가에 있습니다."

방문자 : "거기 세계사도 있어요?"

여기서 "거기"는 **2층 인문학 서가**입니다. 사람 사서는 당연히 기억하지만 LLM은 기본적으로 **기억력이 없습니다**. 매 요청이 독립적이라 이전 질문을 모릅니다. 그래서 **대화 히스토리**를 직접 관리해야 합니다.

모든 대화를 다 기억할 수는 없으니 **최근 N턴만 유지**하는 슬라이딩 윈도우 방식을 씁니다. 사서가 메모장을 들고 있다고 생각하시면 됩니다. 새 메모가 들어오면 가장 오래된 메모를 지우고 최근 5장만 남깁니다.

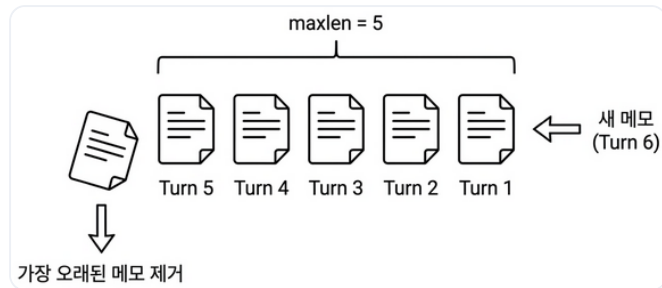


그림 5-4. 6번째 메모가 들어오면 가장 오래된 1번째가 빠집니다. 항상 `maxlen = 5`

Python에서는 `collections.deque(maxlen=5)` 를 쓰면 이 구조가 자동으로 구현됩니다.

실습(conversation.py)

`ex05/src/conversation.py` 는 `deque(maxlen=k)` 위에 얇은 래퍼를 씌워 "이전 대화"를 프롬프트에 넣을 형태로 바꿉니다. `save_turn`, `get_history`, `clear` 세 메서드에 TODO가 있습니다. TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 ex05/src/conversation.py. WindowMemory

```
class WindowMemory:
    def __init__(self, k=5, human_prefix="사용자", ai_prefix="AI 비서"):
        self.k = k
        self.human_prefix = human_prefix
        self.ai_prefix = ai_prefix
        self._turns = deque(maxlen=k)

    def save_turn(self, question, answer):
        # TODO: (question, answer) 튜플을 self._turns에 추가
        # 1. 질문-답변 1턴을 메모장에 저장
        self._turns.append((question, answer))

    def get_history(self):
        # TODO: self._turns의 (question, answer) 쌍을 순회하며
        # 1. 저장된 대화를 "사용자: 질문 / AI: 답변" 텍스트로 변환
        lines = []
        for question, answer in self._turns:
            lines.append(f"{self.human_prefix}: {question}")
            lines.append(f"{self.ai_prefix}: {answer}")
        return "\n".join(lines)

    def clear(self):
        # TODO: self._turns 초기화
```

```
# 1. 메모장 비우기
self._turns.clear()
```

`deque(maxlen=5)` 가 자동으로 오래된 턴을 제거해 주므로 우리가 할 일은 "append → 텍스트 변환 → 비우기" 셋뿐입니다. `ConversationManager` (메모장 관리인) 클래스가

`ex05/src/session_manager.py` 에 정의돼 있어, 세션별로 이 `WindowMemory` 를 하나씩 보관하고, 일정 시간 동안 호출이 없으면 **TTL(Time To Live, 유효 시간)** 로 만료시킵니다.

5.5 서버 실행 + 채팅 UI

두 파일의 TODO를 모두 채웠으면 서버를 띄워 브라우저에서 직접 대화해 봅니다.

터미널 서버 실행

```
python run.py
```

터미널에 `Uvicorn running on http://0.0.0.0:8000` 이 찍히면 준비 완료입니다. 브라우저에서 `http://localhost:8000/chat` 을 열면 채팅 UI가 나타납니다.

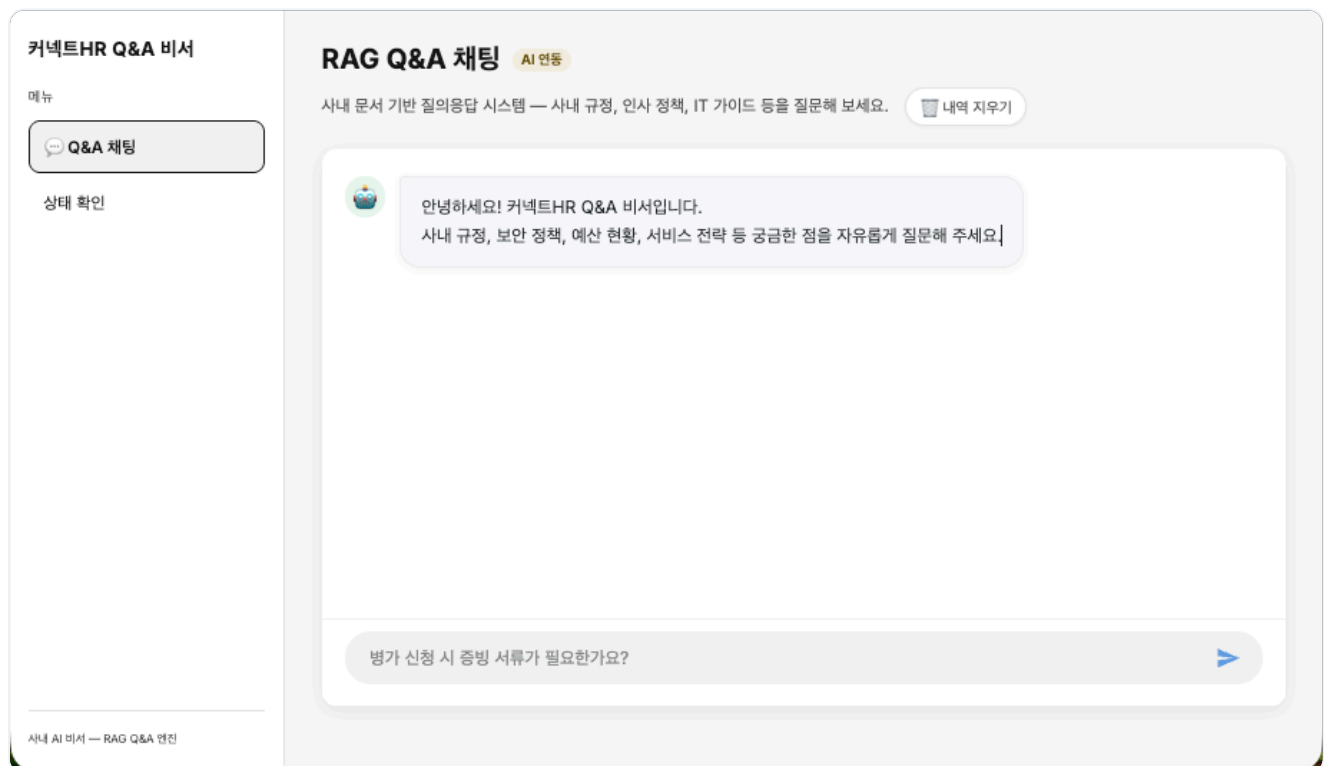


그림 5-5. 커넥트HR 에이전트의 전신. RAG Q&A 채팅 UI

"병가 쓸 때 증빙 서류 필요해요?"라고 물어보면 사내 취업규칙을 근거로 답이 돌아오고, 답변 끝에 출처 파일명이 붙습니다.

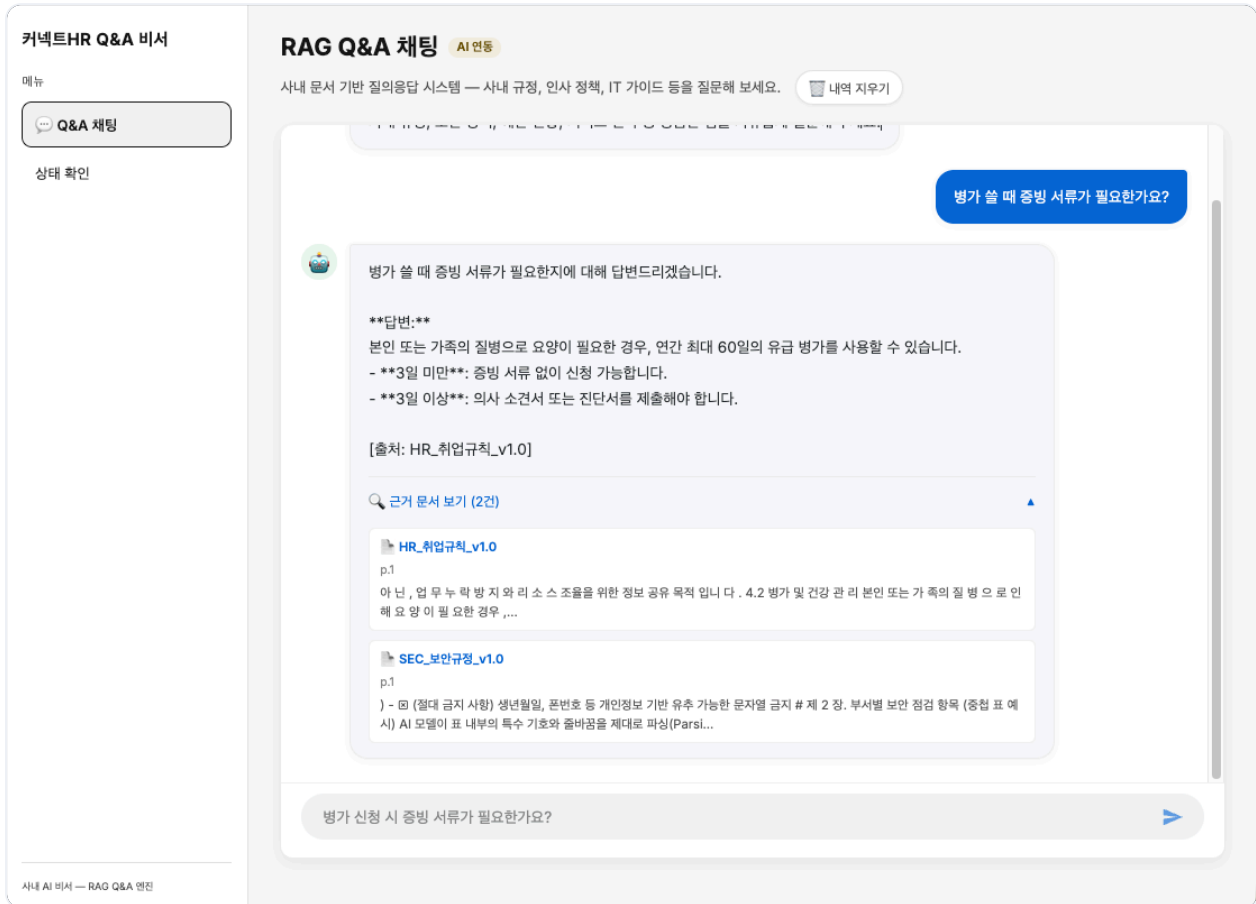


그림 5-6. 답변이 자연어로 돌아오고, 출처까지 명시됩니다

이어서 "그럼 며칠 이상일 때 진단서가 필요해?"라고 물어보면 **이전 대화의 "병가"**를 기억해서 맥락을 이어 답합니다. WindowMemory가 동작한 결과입니다.

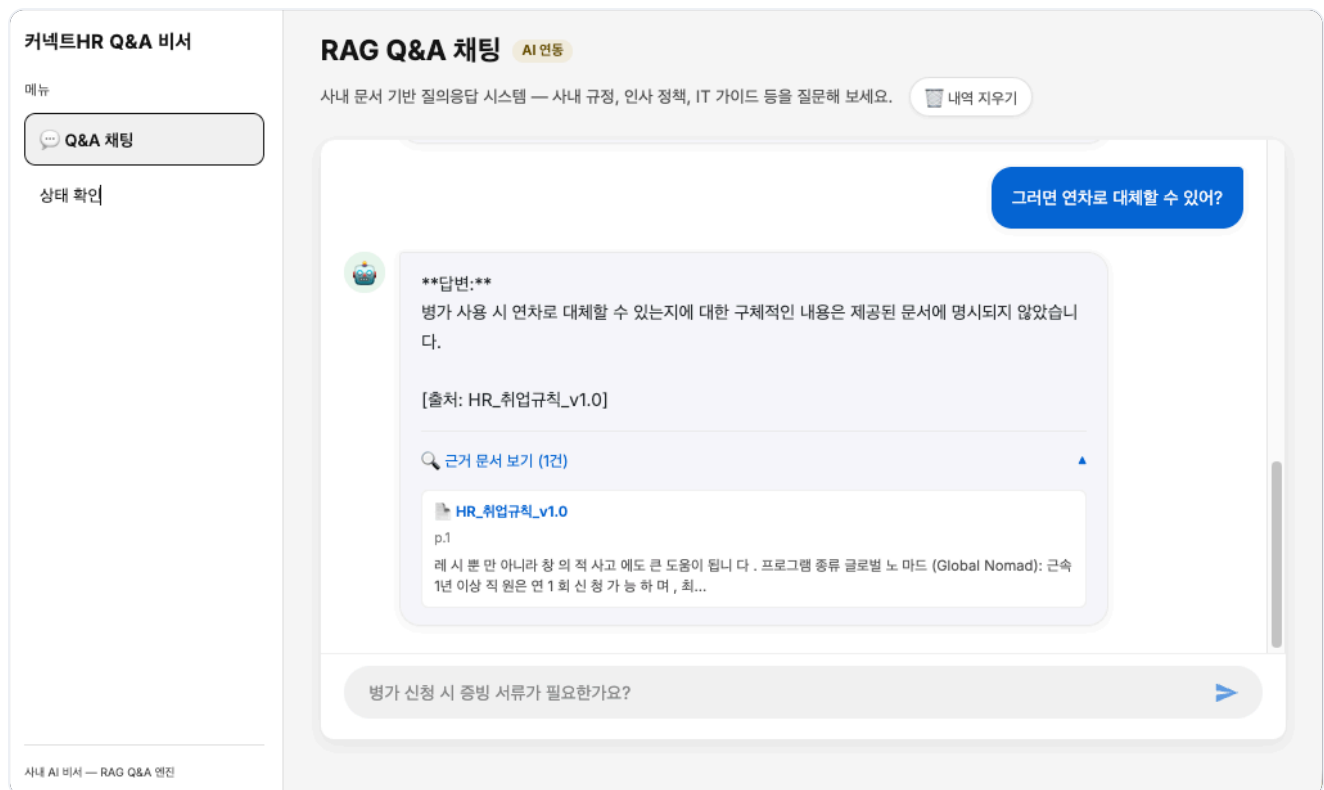


그림 5-7. 멀티턴 대화. 이전 질문을 기억해 "거기", "그럼"을 해석합니다

옆자리에서 동료가 화면을 넘겨다봤습니다.

동료 : "아, 이제 답을 주네요. 아까는 문서 조각만 던지더니."

이번엔 읽을 것도, 해석할 것도 없이 답이 나왔습니다.

오픈이 : "검색 결과를 사용자가 직접 안 읽어도 되네요. 출처 라벨이 자동으로 붙으니 검증도 쉬워졌습니다."

처음으로 **사용자가 쓸 수 있는 형태**의 비서가 생겼습니다. 벡터 검색 결과가 자연어 답변으로 바뀌고, 출처가 붙고, 이어지는 질문의 맥락까지 이해합니다. 다만 "김대리 연차 며칠 남았어?" 같은 **실시간 DB 조회 질문**은 아직 못 풅니다. 챗터 6에서 문서 질문과 DB 질문을 한 에이전트가 동시에 처리하도록 **Tool Calling + ReAct 패턴**을 엮습니다.

이 파이프라인, 다음 챗터에서 어떻게 되는가

지금 만든 LCEL 파이프라인은 여기서 끝이 아닙니다. 다음 챗터에서 이 구조가 **그대로 에이전트 도구 하나** (`search_documents`)로 감싸지고, 옆에 DB 조회 도구(`leave_balance`, `sales_sum`, `list_employees`) 셋이 나란히 놓입니다. 에이전트가 질문을 보고 어떤 도구를 열지 스스로 판단하게 됩니다. **파이프라인 자체는 버려지지 않습니다.**

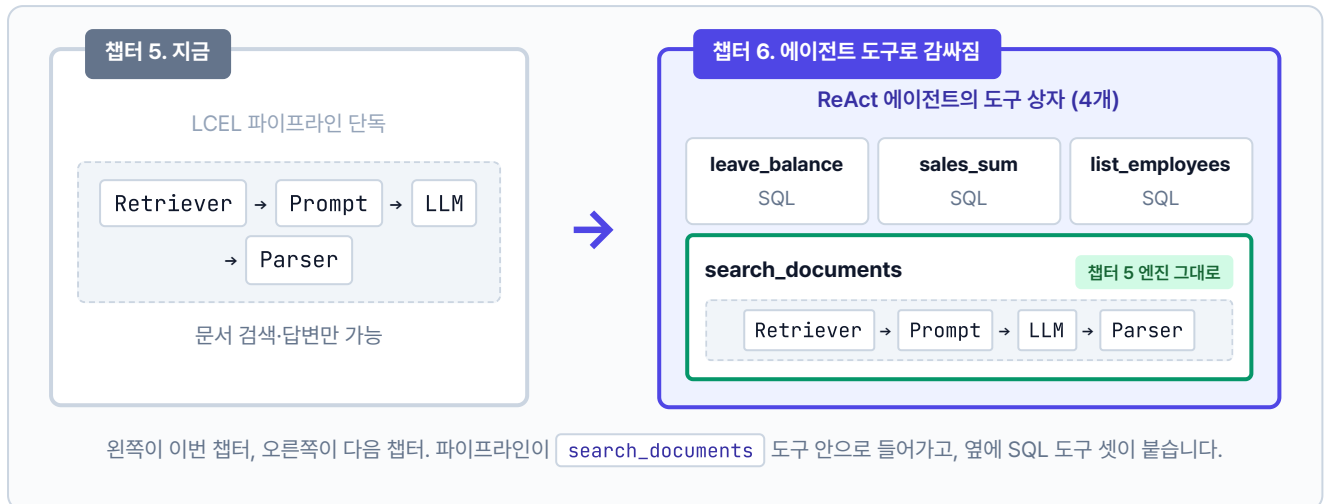
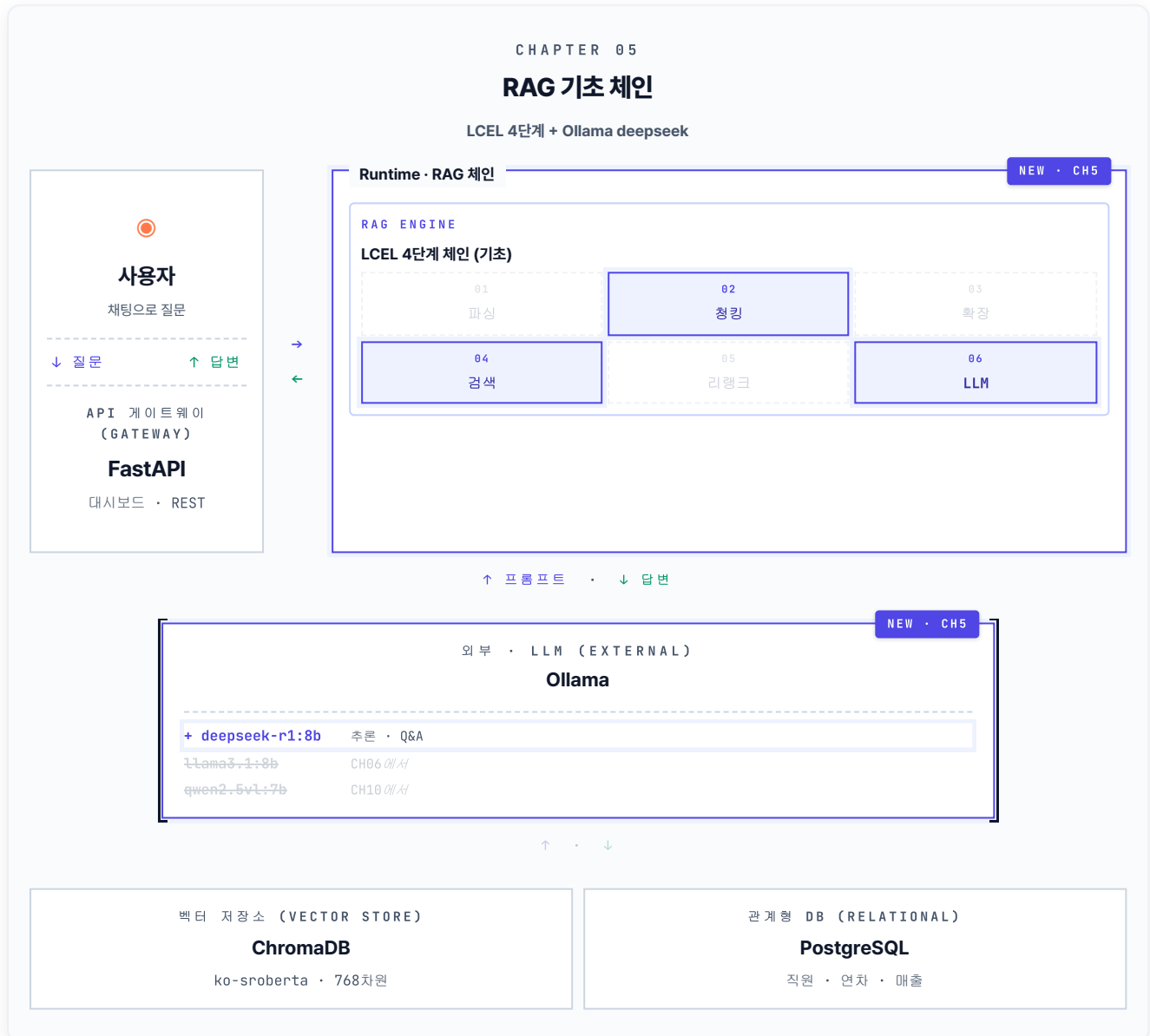


그림 5-8. 이번 챗터의 LCEL 파이프라인은 다음 챗터에서 에이전트 도구가 됩니다

5.6 전체 구성도에서 챗터 5의 자리

CH 05 · 사서가 답해준다 - RAG Engine + LLM NEW



새로: Runtime(RAG Engine) + stage 02·04·06 + LLM(deepseek) · MCP Tools 없어 LLM이 Row 전체

그림 5-9. 챗터 5의 결과. RAG Engine과 Ollama LLM(deepseek)이 새로 자리를 잡았습니다

이 그림이 처음으로 사용자부터 답변까지를 한 화면에 담습니다. 사용자의 채팅이 **API 게이트웨이 (FastAPI)**를 통과해 **Runtime의 RAG Engine**으로 들어가고, 거기서 LCEL 4단계 체인이 ChromaDB와 Ollama(deepseek)를 차례로 부르며 자연어 답변을 만듭니다. 챗터 4까지는

ChromaDB만 외로이 서 있고 검색 결과를 사람이 직접 읽어야 했지만, 이번 챗터에서 사서(RAG Engine)와 통역사(LLM)가 자리잡으면서 "질문 → 답변"의 한 줄 경로가 닫혔습니다.

NEW 표시가 붙은 칸 안의 stage 02·04·06이 이번 챗터에서 켜 자리입니다. **stage 02 검색**(Retriever)이 ChromaDB에서 의미가 가까운 청크를 가져오고, **stage 04 프롬프트 조립**(Prompt + LLM)이 출처를 강제하며 답변을 생성하고, **stage 06 메모리**(WindowMemory)가 최근 다섯 턴을 기억해 후속 질문을 이어 받습니다. 비어 있는 stage 01(파싱)·03(질의 변환)·05(리랭크)는 챗터 8~10 튜닝에서 채워질 빈자리입니다.

ch-note에 적힌 "MCP Tools 없어 LLM이 Row 전체"는 이번 챗터의 한계를 짚어 줍니다. LLM이 도구 없이 한 가지 일(문서 검색 답변)만 한다는 뜻입니다. 다음 챗터에서 **ReAct 에이전트**가 등장하면 지금 만든 LCEL 파이프라인이 그대로 도구 하나(`search_documents`)로 들어가고, 그 옆에 **SQL 도구 셋**(`leave_balance` · `sales_sum` · `list_employees`)이 나란히 붙어 사서가 장부실 열쇠까지 켜 "하이브리드 사서"가 됩니다. 그림 5-8에서 미리 그려 두었던 변환이 챗터 6에서 코드로 실현됩니다.

파이프라인은 그대로, 도구로 감싸진다는 게 챗터 6의 그림입니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"사서의 업무 순서"	LCEL 파이프라인	LangChain Expression Language. 파이프(<code> </code>)로 Retriever → Prompt → LLM → Parser를 연결
"근거를 대"	출처 강제(Source Grounding)	프롬프트로 "제공된 문서에서만 답하고 출처 명시"를 강제하는 기법
"5장짜리 메모장"	WindowMemory	최근 N턴 대화만 유지하는 슬라이딩 윈도우. <code>deque(maxlen=k)</code> 기반
"메모장 관리인"	ConversationManager	세션별 WindowMemory를 보관하고 TTL로 만료 세션을 정리하는 클래스

"입력 → 출력 연결"

파이프 연산자(`|`)

`A | B` 는 A의 출력을 B의
입력으로 전달. Unix 파이프와 같은
개념

이것만은 기억하자

- **벡터 검색은 재료, LCEL은 요리입니다.** 검색 결과를 던져 주는 게 아니라 읽고 답해 주는 체인을 엮어야 사용자가 쓸 수 있는 비서가 됩니다.
- **출처 강제는 환각의 안전망입니다.** "제공된 문서에서만 답하라 + 출처를 명시하라" 한 줄이 거짓말을 크게 줄여 줍니다. 답변에 출처 파일명을 항상 붙이세요.
- **기억 없는 LLM에게 메모장을 쥐여 줍니다.** WindowMemory(k=5)면 일상 대화에 충분합니다. 챗터 6에서는 에이전트가 도구를 직접 골라 "문서 질문"과 "DB 조회 질문"을 한 번에 처리합니다.